

The Concurrency Trap on GPU Infra

Why the KV cache — not the model weights — decides how many users your GPU can serve

Client: Architect Briefing · Philips

Trainer: Advanced Agentic AI Bootcamp

Date: Deployment lesson



THE CORE LESSON

Weights Fit. Concurrency Doesn't.

Every architect sizing a GPU deployment must understand this: at real context lengths and real concurrent load, the KV cache consumes more memory than the model itself.



The Mistake That Sinks Deployments

The naive question: "The model is 27 GB and my GPU has 80 GB — it fits, ship it." **The reality:** that only sizes for ONE user. Every concurrent user needs their own KV cache — and that grows fast.

What actually lives in your GPU memory:

- **Model weights — fixed. Load once, shared by everyone. (e.g. ~27 GB for a 27B FP8 model)**
- **KV cache — PER REQUEST. Grows with context length × concurrent users. This is the variable that bites.**
- During generation the model stores key/value vectors for every token so it doesn't recompute them — these accumulate for the whole request and only free when it finishes.
- At short context it's negligible. At long context with real concurrency it becomes the dominant memory consumer.



THE EVIDENCE

The Numbers Every Architect Should See

Concrete VRAM figures from Spheron's KV-cache optimization guide (Llama 3.1 70B, BF16) — the shape of the problem is the same for any model.



KV Cache Grows With Context × Users

KV cache memory — Llama 3.1 70B at BF16 (illustrative; weights alone are ~140 GB):

Context length	1 user	8 concurrent users
4,096 tokens	~1.3 GB	~10.7 GB
16,384 tokens	~5.4 GB	~43 GB
32,768 tokens	~10.7 GB	~86 GB
131,072 tokens	~42.9 GB	~343 GB

Read this

- At 32K / 8 users, the KV cache (~86 GB) exceeds the model weights.
- At 128K / 8 users it's ~2.4× the model size.
- One 128K request alone needs ~42.9 GB — nearly a whole 80 GB card.
- This is why 'it fits' for one user tells you nothing about concurrency.

Source: Spheron, KV Cache Optimization guide (2026). Figures illustrative for Llama 3.1 70B.



The Formula Behind It

KV cache memory

$$\text{KV_bytes} = 2 \times L \times H_{\text{kv}} \times D \times S \times B \times \text{bytes}$$

L = transformer layers

H_{kv} = key-value heads (after GQA)

D = head dimension

S = sequence length (tokens)

B = concurrent batch size

$$\text{bytes} = 2 \text{ (BF16)} \cdot 1 \text{ (FP8)} \cdot 0.5 \text{ (FP4)}$$

The two levers you control

- **S (context) and B (users) multiply — double both and cache 4×**
- bytes: FP8 halves it, FP4 quarters it
- L , H_{kv} , D are fixed by the model you chose

Why architecture already helps — GQA

- **Grouped-Query Attention (GQA) shares KV heads across query heads — Llama 3 70B uses 8 KV heads for 64 query heads = 8× smaller cache than plain multi-head attention.**
- Qwen 2.5 72B: also 8× · Llama 3.1 8B: 4×. Choose GQA-native models for long-context serving.
- This is baked into modern models — you don't configure it, but it's why the numbers above aren't even worse.



THE PLAYBOOK

Five Techniques That Compound

The Spheron guide shows these stack: from a naive baseline of 4 concurrent users to 8× more on the same GPU. Optimize the cache first, then size the GPU to the optimized budget.



The Optimization Stack

1 · PagedAttention

Pages the cache like OS virtual memory — no wasted pre-allocation. On by default in vLLM.

2–4× more concurrent users

2 · FP8 KV cache

Store KV at 8-bit. `--kv-cache-dtype fp8` (H100/A100). Negligible quality loss.

Halves cache vs BF16

3 · NVFP4 KV cache

4-bit KV — Blackwell only (B200/B300). Validate on reasoning tasks first.

Further 50% vs FP8

4 · CPU offload / swap

`--swap-space 32` uses system RAM as overflow during spikes (adds 2–10ms).

Prevents request rejection

5 · Prefix caching (LMCache)

Reuse shared prefixes (system prompt, RAG doc). `--enable-prefix-caching`.

TTFT ~11s → ~1.5s



How the Gains Stack (Llama 3.1 70B, H100)

Each technique builds on the previous — same GPU, more users:

Configuration	Concurrent users	KV cache VRAM
Naive pre-alloc, BF16 KV	4	~40 GB
+ PagedAttention (default)	8	~22 GB
+ FP8 KV cache	16	~11 GB
+ Prefix caching (LMCache)	16 · TTFT 1.5s	~11 GB
+ NVFP4 KV (Blackwell)	32	~5.5 GB

8× more users on the same GPU. These are estimated/illustrative figures from published vLLM & LMCache data, not independently reproduced — always benchmark on your own hardware and request mix before committing.



What It Looks Like in vLLM

vllm serve — the flags that matter

```
python -m vllm.entrypoints.openai.api_server \
  --model <your-model> \
  --gpu-memory-utilization 0.90 \      # cache pool
  --max-model-len 32768 \             # cap context!
  --kv-cache-dtype fp8 \              # halve cache
  --enable-prefix-caching \           # reuse prefixes
  --swap-space 16                     # burst overflow

# PagedAttention + Flash Attention: on by default
# NVFP4 (--kv-cache-dtype nvfp4): Blackwell ONLY
#   - will error / misbehave on H100 or A100
```

Architect checklist

- Cap --max-model-len to your REAL need, not the model's max
- FP8 KV on H100/A100 — safe, big win
- NVFP4 only on Blackwell — never on Hopper/Ampere
- --gpu-memory-utilization 0.90 (0.92–0.95 if tight)
- swap-space for bursty traffic (needs 64+ GB host RAM)
- Size the GPU to the OPTIMIZED budget, not the naive one



FOR PHILIPS

Why This Matters for Our Deployments

The bootcamp labs run ~21 concurrent agents. Agentic workloads make the concurrency problem worse — and this is exactly the sizing trap to avoid.



Agentic Workloads Amplify This

Why agents make it worse

- **21 concurrent participants = 21 KV caches at once**
- Agents run multi-step loops — many calls per task
- Reasoning / thinking modes generate long chains → big cache per request
- Red-team & capstone are the peak-concurrency moments — exactly when a mis-sized GPU queues
- Tool-calling context accumulates across the loop

How we size correctly

- **Weights (FP8 ~17–27 GB) leave headroom — that was never the constraint**
- Budget KV cache for 21 users at the labs' real context length
- FP8 KV cache on the H100/A100 by default
- Cap max-model-len to what the labs actually use
- PagedAttention (vLLM default) is doing the heavy lifting for us



One Sentence to Remember

"Model weights tell you if it loads. The KV cache tells you how many people it can serve."

Optimize the cache first — PagedAttention, FP8, prefix caching — then size the GPU to that optimized budget.

Grounded reference — read the full guide:

Spheron — KV Cache Optimization: Serve 10× More Users per GPU (2026)

<https://www.spheron.network/blog/kv-cache-optimization-guide/>

Related, from the same source: GPU Memory Requirements for LLMs · GQA vs MHA KV cache savings · vLLM Production Deployment.

Size for Concurrency

The weights fit. The question is always: how many users at what context length?
Answer that with the KV cache math — before you provision.
